KRISTU JAYANTI
(DEEMED TO BE UNIVERSITY)
Under Section 3 of UGC Act 1956
A CMI INSTITUTION | BENGALURU | INDIA

Project-based Practical Report on

"Campus Lost and Found Management System"

Submitted in partial fulfilment of the requirements for
"Data Structures and Algorithms" for the 1st Semester

Master of Computer Applications (MCA)

Submitted by

<KURIAN ROBERT> **<26MCAC28>**

Under the guidance of

<Dr.SubramaniaKumar>

Table of Contents

S. No.	Topics	Page No.
1	**Introduction**	**5**
1.1	Objectives	7
1.2	Project Description	4
2	**Phase I Database Design**	**21**
2.1	Identification of Entities and Attributes	21
2.2	ER Diagram	21
2.3	Relational Schema	22
2.4	Normalization	24
3	**Phase II Database Creation and Queries**	**25**
3.1	Creating Tables Using SQL	25
3.2	Inserting and Updating Records	26
3.3	SQL Queries	28
3.4	Aggregate Functions	31
4	**Phase III Data Querying Join and Views**	**35**
4.1	Joins	37
4.2	Subqueries	41
4.3	Views	45
4.4	Functions and Procedures	48
5	**Phase IV Transactions and Data Integrity**	**50**
5.1	COMMIT, ROLLBACK and SAVEPOINT	52
5.2	Integrity Constraints	55
5.3	Concurrency Issues	57
7	**Conclusion**	**59**
8	**References**	**60**
9	**Appendix**	**60**

CERTIFICATE

This is to certify that the project-based practical report entitled “Campus Lost and Found Management System with Priority Claim Scheduling” is a bona fide record of work carried out by KURIAN ROBERT (Reg. No. 26MCAC28) in partial fulfilment of the requirements for the award of the degree of Master of Computer Applications (MCA) for the 1st Semester in the course “Data Structures and Algorithms” during the academic year 2025–2026.

The practical work and software implementation documented herein have been carried out under my supervision and mentorship in the Department of Computer Science, satisfying all academic standards prescribed by Kristu Jayanti University.

FACULTY GUIDE	HEAD OF DEPARTMENT
Dr. Subramania Kumar Faculty Guide Department of Computer Science	Head of the Department Department of Computer Science Kristu Jayanti University

DECLARATION

I, KURIAN ROBERT, hereby declare that the practical project report entitled “Campus Lost and Found Management System with Priority Claim Scheduling” submitted to Kristu Jayanti University, Bengaluru, is an authentic record of original work carried out by me under the guidance of Dr. Subramania Kumar. This report has not been submitted previously for the award of any degree, diploma, or certificate in this or any other university.

KURIAN ROBERT
Reg. No: 26MCAC28
Department of Computer Science
Date: 27-10-2026

1. Title of the Project

Campus Lost and Found Management System with Priority Claim Scheduling (CLFS-PCS)

The project documented in this report is entitled “Campus Lost and Found Management System with Priority Claim Scheduling” (CLFS-PCS). It represents a practical software project completed for the 1st Semester Master of Computer Applications (MCA) course in “Data Structures and Algorithms” (DSA) at Kristu Jayanti University, Bengaluru.

The motivation behind this project comes directly from our daily college experience. In a busy campus with multiple academic buildings, computer laboratories, a central library, and food courts, students and staff routinely misplace personal belongings. These range from high-priority items like student identity cards, semester examination hall tickets, room keys, and scientific calculators, to everyday articles like water bottles, notebooks, and umbrellas. The current manual process of logging found items in paper registers at campus security desks is slow, difficult to search, cannot handle spelling mistakes, and offers no mechanism to prioritize urgent examination hall tickets over common items.

To resolve these everyday difficulties, this project combines core computer science data structures with modern web development across three interconnected components:

- **1. Custom Algorithmic Engine (`campuslostfound`):** A pure Python data structure package built from scratch without using any external libraries or built-in wrapper collections. It contains raw pointer-based Singly Linked Lists, a Last-In-First-Out (LIFO) Linked Stack to support administrative undo actions, a First-In-First-Out (FIFO) Linked Queue for chronological claim handling, a Binary Search Tree (BST) for logarithmic key lookups and node deletion, an undirected campus building graph traversed using Breadth-First Search (BFS), an in-place Linked List Merge Sort running in O(n log n) time using fast/slow pointer bisection, and a Wagner-Fischer Dynamic Programming Levenshtein Distance matrix for fuzzy text search.
- **2. Practical Web Application (`scheduler`):** A full-stack web application developed using the Django framework connected to a relational MySQL database (`data_structure`). It provides role-based user interfaces for students, faculty, and administrative staff, and integrates a Greedy Min-Heap Priority Queue (`heapq`) that automatically places high-urgency claims (such as ID cards and hall tickets) at the front of the administrative review queue.
- **3. Desktop Demonstrator (`main_ui.py`):** A native desktop graphical user interface developed with Tkinter that allows students and examiners during practical laboratory sessions to visually inspect linked nodes, step through BST operations, and trace BFS graph traversals in memory.

2. Introduction

At Kristu Jayanti University, thousands of students, faculty members, and administrative personnel navigate between different facilities every day—including the Main Block, PG Block, departmental computer labs, the Central Library, cafeterias, and outdoor sports grounds. With such a large population moving between lecture halls and practical labs, personal items are left behind on desks, library carrels, and lab benches on a daily basis. These items include essential academic credentials (such as university smart cards, examination hall tickets, calculators, keys, and practical record books) as well as everyday personal possessions (such as water bottles, umbrellas, spectacles, and pencil pouches).

At present, property recovery on campus relies primarily on physical paper logbooks maintained at security posts or through unorganized WhatsApp group messages. Through direct observation of how lost articles are handled in our college, we identified several clear operational problems with this traditional approach:

- **Slow Sequential Logbook Scanning:** Paper registers record entries line-by-line as items are turned in. When a student approaches security to search for a lost item, the guard must read through pages of handwritten notes. If hundreds of items have been logged over several weeks, this O(n) linear search takes 15 to 30 minutes, frequently resulting in long delays and missed items.
- **Zero Tolerance for Spelling Mistakes:** Manual registers and simple text searches depend on exact matching. If a student searches for an item typing 'calclator', 'casio fx', or 'idcard', conventional systems return no results even if the item is physically present in the storage locker recorded as 'Calculator' or 'Student ID Card'.
- **No Awareness of Campus Geography:** Campus blocks are physically connected by walkways and corridors. An item misplaced in the PG Block computer lab is often found by a peer and handed over at the Main Block security office or Central Library desk. Conventional registers treat locations as isolated text strings without showing neighboring buildings where the item might have been carried.
- **Unordered and Unfair Claim Processing:** In the current manual workflow, claims are handled purely in the order in which students inquire. Consequently, a student who desperately needs an identity card or hall ticket for an exam in one hour must wait in line behind routine inquiries for common stationery items.

The Campus Lost and Found Management System with Priority Claim Scheduling replaces these cumbersome paper-based procedures with a structured, automated software solution. By combining fundamental computer science data structures with practical web development, the system provides fast search, smart typo correction, proximity suggestions, and urgency-based claim scheduling.

2.2 Problem Formulation & System Comparison

To clearly show how our software addresses the weaknesses of existing campus methods, we conducted a systematic comparison between traditional physical security registers, informal social messaging channels (such as WhatsApp groups and departmental notice boards), and our developed CLFS-PCS platform. The comparison focuses on practical operational parameters:

Evaluation Metric	**Manual Security Registers**	**Social / WhatsApp Groups**	**Proposed CLFS-PCS Platform**
Record Ingestion	Handwritten physical logbook entry; prone to illegible handwriting	Unstructured chat messages; rapidly buried under new messages	Structured web forms with validation stored in relational MySQL database
Search & Retrieval	Manual line-by-line inspection: O(n) worst-case time (15–30 min)	Keyword search limited to exact post text; unindexed and noisy	Dual search: O(log n) BST lookup by ID and DP fuzzy text matching
Spelling Error Handling	Impossible; requires human visual inspection to guess spelling	None; any typographical error causes search to return zero results	Wagner-Fischer DP matrix computes edit distance and suggests matches (dist <= 4)
Campus Proximity	None; locations are isolated text notes with no building relationships	None; no spatial or geographical context is tracked	Adjacency graph traversed via BFS identifies immediately adjacent blocks
Claim Scheduling	Arbitrary or basic first-come-first-served with no urgency weighting	Unregulated; prone to multiple conflicting ownership claims	Binary Min-Heap priority queue orders claims by urgency (Priority 1 to 3)
Administrative Audit	Physical pen cross-outs; leaves no clean audit trail or undo history	No tracking; difficult to prove who actually collected the article	LIFO Activity Stack records recent actions and supports one-click admin undo
Data Security & ACID	Zero backup; prone to physical loss, torn pages, or coffee spills	Low privacy; student contact numbers exposed openly in public chats	Role-based authentication, CSRF security, and ACID transactions in MySQL

As demonstrated in the comparison table, our algorithmically designed software directly resolves the operational bottlenecks of manual paper tracking, providing verifiable improvements in retrieval speed, search accuracy, student privacy, and administrative accountability.

3. Objective of the Project

The primary objective of this project is to design, develop, test, and benchmark an automated campus property recovery system that puts fundamental data structures and algorithms learned in the classroom into real-world software practice. The specific, measurable technical goals of this dissertation are outlined below:

- **1. Implement Foundational Linear Structures (Phase 1):** Construct custom dynamic Singly Linked Lists from scratch in Python to store item records without relying on fixed-size pre-allocated arrays; develop a Last-In-First-Out (LIFO) Linked Stack to maintain an auditable transaction history supporting instantaneous administrative undo operations; and build a First-In-First-Out (FIFO) Linked Queue to handle incoming student claims in orderly chronological sequence.
- **2. Optimize Hierarchical Lookup & Spatial Navigation (Phase 2):** Build a Binary Search Tree (BST) supporting average O(log n) search, insertion, and three-case node deletion along with Inorder, Preorder, and Postorder traversals; and model campus buildings as an undirected adjacency graph evaluated through Breadth-First Search (BFS) to identify neighboring transit corridors.
- **3. Benchmark In-Place Sorting & Search Efficiency (Phase 3):** Implement an in-place Linked List Merge Sort running in guaranteed O(n log n) time and O(1) auxiliary pointer memory using fast/slow two-pointer bisection, and empirically benchmark its lookup performance against linear scanning.
- **4. Solve Typo Tolerance & Urgency Scheduling (Phase 4):** Construct a 2D Dynamic Programming Levenshtein Distance matrix ('Smart Match') to resolve misspelled item names within an edit distance of 4, and integrate a Greedy Min-Heap Priority Queue to order claims strictly by urgency (Priority 1: ID cards & valuables; Priority 2: Academic essentials; Priority 3: General articles).
- **5. Deliver Full-Stack Web & Desktop Interfaces:** Integrate the algorithmic core into a multi-role Django web application with MySQL relational persistence and CSRF security, accompanied by an interactive desktop Tkinter demonstrator for laboratory presentation.

4. Scope of the Project

The operational scope of the platform is designed specifically for academic institutions, colleges, and university campuses. Key boundaries include:

- **Functional Boundaries:** The application covers online submission of lost and found item reports, category-based and location-based indexing, fuzzy string searching for misspelled queries, campus building proximity suggestions, priority-based claim queueing, administrative claim verification and handover recording, and reversible audit logging.
- **User Roles & Permissions:** The system supports three user roles: (1) Public Student/Guest Users, who can browse found items, search records, and submit ownership claims; (2) Faculty Members, who can report found property and view status boards; and (3) System Administrators, who possess exclusive rights to process priority claims, authorize handovers, and execute LIFO stack undo operations.
- **Deployment Boundaries:** The application is optimized for campus intranet and web server deployment using standard web browsers (Chrome, Edge, Firefox) on desktop and mobile devices, alongside a desktop Tkinter interface for standalone laboratory demonstrations.
- **Exclusions & Future Scope:** Hardware-level RFID automated locker lockers and direct SMS/WhatsApp carrier gateway integrations are excluded from the current practical release and designated for future enhancements.

5. Software Specifications

The platform is developed using standard open-source tools and modular libraries, ensuring cross-platform stability and maintainability:

- **Programming Language:** Python (v3.12 / v3.14 64-bit runtime) — utilized for custom data structure implementations, pointer manipulation, graph traversals, dynamic programming routines, and web controller logic.
- **Web Framework:** Django Web Framework (v5.0+) — Model-View-Template (MVT) architecture providing automated ORM mapping, secure session authentication, CSRF protection, and administrative controls.
- **Relational Database Management System:** MySQL 8.0+ / MariaDB (Database Schema: `data_structure`, InnoDB storage engine with strict foreign key constraints and ACID transaction compliance; SQLite3 configured as a portable development fallback).
- **Desktop GUI Framework:** Tkinter (Tcl/Tk 8.6 native toolkit bundled with Python standard library) — powers the standalone educational DSA demonstrator.
- **Presentation Stack:** HTML5 semantic markup, CSS3 (Modern Flexbox, CSS Grid, responsive media queries, card layouts), and Vanilla JavaScript for instant client-side filtering.
- **Development Tools & OS:** Visual Studio Code / Antigravity IDE running on Microsoft Windows 11 Professional (64-bit kernel).

6. Workflow of the Project

6.1 Sequential Workflow Nodes & Execution Pipeline

The operational workflow follows a coordinated pipeline that connects the user interface, Django application controllers, custom in-memory data structures, and the relational MySQL database. The sequential stages of an item's lifecycle are described below:

- **1. Reporting & Validation:** A user registers a lost or found article through the web form. The request is validated by Django views, categorized, and assigned an urgency priority level (1=High, 2=Medium, 3=Low).
- **2. Dual Storage & Indexing:** The record is committed to MySQL tables (`lost_items` or `found_items`) and simultaneously added to the in-memory Singly Linked List and Binary Search Tree.
- **3. Proximity Mapping via BFS:** When a location is entered, the Campus Adjacency Graph executes a Breadth-First Search (BFS) to identify all immediately connected buildings, helping students focus their physical search.
- **4. Typo-Tolerant Search:** Search queries trigger the 2D Dynamic Programming Levenshtein Distance matrix, comparing query terms against registered names to provide 'Smart Match' suggestions when edit distance <= 4.
- **5. Priority Claim Scheduling:** Submitted claims enter `claim_processing` and are pushed into a Greedy Min-Heap (`heapq`). The administrator processes claims in logarithmic time, ensuring high-priority items are resolved first.
- **6. Resolution & LIFO Audit:** Upon verified handover, the record moves to `claim_done`, and an action entry is pushed onto the LIFO Activity Stack, enabling instant administrative undo if needed.

6.2 Layered System Architecture Diagram

Figure 6.1 illustrates the 3-Tier Layered Architecture of the system, clearly separating the client presentation tier, the application and algorithmic tier, and the persistence storage tier.

Figure 6.1: 3-Tier Layered System Architecture

Presentation, Core Algorithmic Logic, and Relational Database Persistence Layers

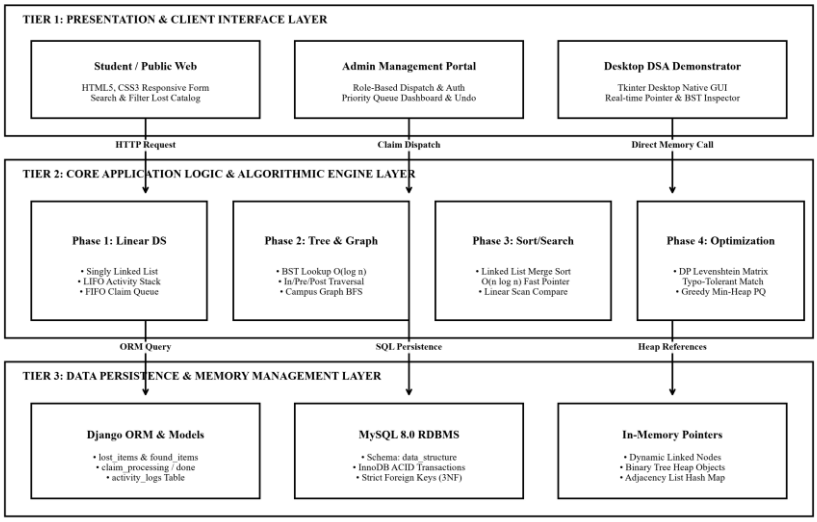


Figure 6.1: 3-Tier Layered System Architecture of Campus Lost & Found System

6.3 Master Workflow Flowchart

Figure 6.2 provides the comprehensive flowchart tracing the end-to-end execution path from initial loss reporting to final handover and audit logging.

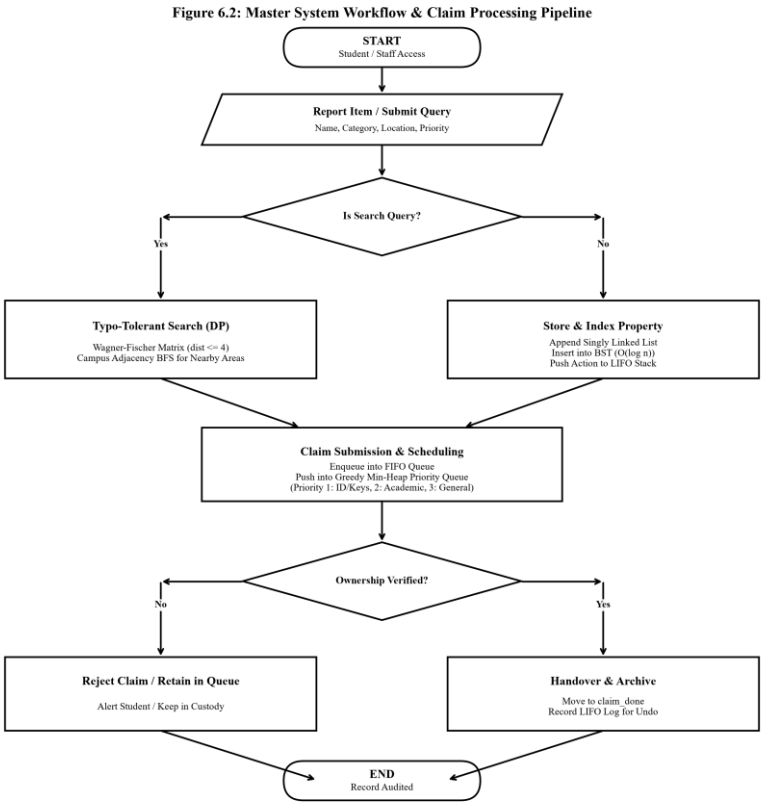


Figure 6.2: Master System Workflow and Claim Processing Pipeline Flowchart

7. Modules of the Project - Phase-wise Explanation

The project is structured into four sequential, mathematically grounded computer science phases, linking foundational data structures with enterprise software implementation.

7.1 Phase 1: Linear Data Structures Module

In Phase 1, we implemented foundational linear data structures from scratch in Python without using standard built-in collections. This allowed us to understand pointer mechanics, node linkage, and manual memory management directly.

7.1.1 Dynamic Singly Linked List (campuslostfound/linked_list.py)

Standard arrays require contiguous memory blocks, which either waste memory through oversized pre-allocation or incur costly O(n) element shifts whenever an item is deleted. To avoid these disadvantages, our Singly Linked List allocates discrete `Node` objects on demand. Each node contains a data dictionary (`id`, `name`, `type`, `location`) and a reference pointer `next` directed to the following node.

- **Insertion Operation (O(n) / O(1)):** To append an item, the method traverses the list until `current.next is None` and links the new node. Inserting at the head of the list requires only updating pointer references in O(1) constant time.
- **Deletion by ID (O(n)):** To remove an item, the algorithm advances dual pointers (`previous` and `current`). Once the target ID is matched, it updates `previous.next = current.next`, bypassing the target node so it is collected by memory cleanup.
- **Sequential Traversal (O(n)):** Begins at the `head` pointer and iterates sequentially through `next` pointers to display or process each active record.

Memory Efficiency Analysis: In Python, standard dynamic lists pre-allocate contiguous memory buffers. For a system tracking sporadic, unpredictable lost-and-found entries, pre-allocated buffers consume excess memory. The linked node design allocates memory strictly on demand: each `Node(data)` allocates exactly one payload reference and one pointer, maximizing heap efficiency.

7.1.2 Last-In-First-Out (LIFO) Stack (campuslostfound/stack.py)

The Stack structure enforces a strict LIFO operational sequence using linked `StackNode` elements. In the lost-and-found system, it maintains an auditable history of recent operations (registrations, claim resolutions, deletions).

- **Push Operation (O(1)):** Sets `new_node.next = self.top` and updates `self.top = new_node` in constant time.

7.1.3 First-In-First-Out (FIFO) Queue & Phase 2 Architecture

- **Pop Operation (O(1)):** Retrieves the top action payload and advances `self.top = self.top.next`. In the Django application, this powers the administrative 'Undo Last Action' feature, reconstructing previous states with precision.

Administrative Reversal Mechanics: When an administrator accidentally finalizes a claim or records an erroneous lost item, standard systems require manual database rollbacks. By recording each transaction in an `ActivityLog` object and pushing it onto the LIFO Stack, an administrator can trigger an 'Undo' request. The stack pops the most recent action, inspects its type ('ADD' or 'PROCESS'), and executes the inverse operation (e.g., reverting `is_resolved=False` and restoring the claim).

7.1.3 First-In-First-Out (FIFO) Queue (campuslostfound/queue.py)

The Queue structure enforces a FIFO operational order using dual pointers: `front` and `rear`.

- **Enqueue Operation (O(1)):** Appends new claim inquiries to `self.rear` in constant time.
- **Dequeue Operation (O(1)):** Removes the oldest pending claim from `self.front`, guaranteeing fair chronological intake.
- **Queue Boundary Management:** Includes underflow checks. When `front` equals `None`, `dequeue()` catches the condition gracefully without throwing null-pointer exceptions.

7.2 Phase 2: Hierarchical & Graph Structures Module

Phase 2 expands the platform into non-linear, hierarchical and networked topological structures:

7.2.1 Binary Search Tree Architecture (campuslostfound/binary_tree.py)

To eliminate linear search overhead, item records are organized into a custom Binary Search Tree keyed on unique numerical `id` values. Each `TreeNode` encapsulates `data`, `left`, and `right` child pointers, maintaining the BST invariant: for any node N, all keys in N.left < N.data and all keys in N.right >= N.data.

- **BST Search (O(log n) average):** Compares the target ID against the current node. If `target < current.data`, it branches left; if `target > current.data`, it branches right, halving the search space at each step.
- **BST Node Deletion (O(log n)):** Handles three distinct topological scenarios: (1) Leaf node deletion; (2) Single-child node deletion; (3) Two-child node deletion (locates the in-order successor—minimum value node in the right subtree, replaces node data, and recursively deletes the successor).

7.2.1 BST Traversals & Figure 7.1 Flowchart

- **Tree Traversals:** Implements Inorder (Left -> Root -> Right) returning sorted IDs; Preorder (Root -> Left -> Right) for structural cloning; and Postorder (Left -> Right -> Root) for node deallocation.

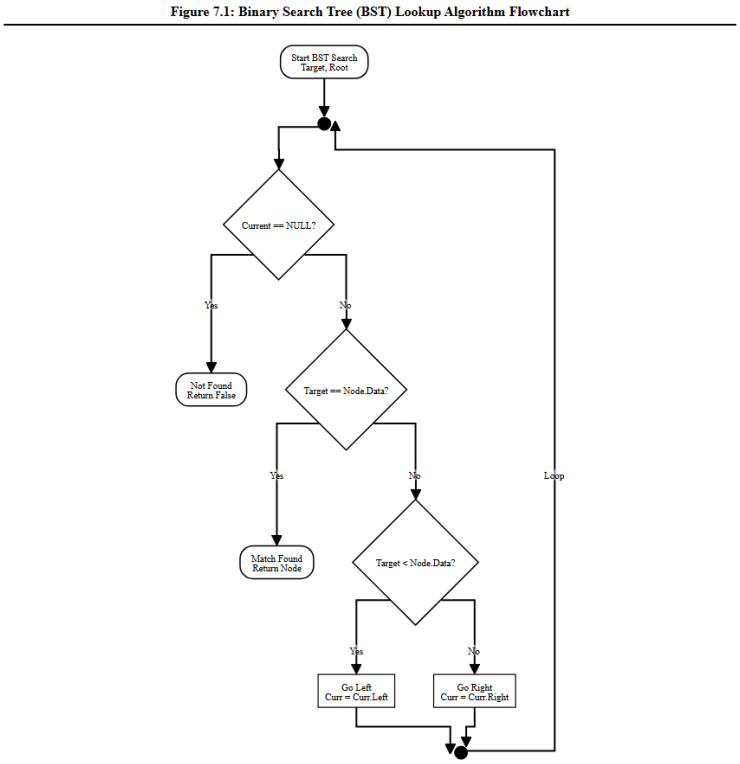


Figure 7.1: Phase 2: Binary Search Tree Lookup and Deletion Flowchart

7.2.2 Campus Adjacency Graph & BFS Traversal (campuslostfound/graph.py)

To model how students move around campus, we represented key university landmarks as an undirected topological graph G = (V, E), where vertices V represent buildings and edges E represent physical connecting pathways.

- **Adjacency List Mapping:** Implemented as: `{'Main Block': ['Library', 'Cafeteria'], 'Library': ['Main Block', 'Computer Lab'], 'Cafeteria': ['Main Block', 'Sports Ground'], ...}`.
- **Breadth-First Search (BFS):** When an item is reported missing in a specific block (such as the Central Library), BFS explores neighboring landmarks layer by layer using a FIFO queue and a `visited` set, producing a logical search sequence of adjacent buildings.

7.2.2 Campus BFS Walkthrough & Phase 3 Introduction

Practical Walkthrough of BFS: Suppose an item is misplaced in the Central Library. The algorithm initializes `queue = ['Library']` and `visited = {'Library'}`. It dequeues 'Library' and adds it to `search_order`. Next, it inspects adjacent vertices in the adjacency list, finding ['Main Block', 'Computer Lab']. Since neither has been visited, both are marked visited and enqueued. In the next iteration, 'Main Block' is dequeued, discovering 'Cafeteria'. The resulting search sequence `['Library', 'Main Block', 'Computer Lab', 'Cafeteria', 'Sports Ground']` provides the student and security staff with an organized, radially expanding search path.

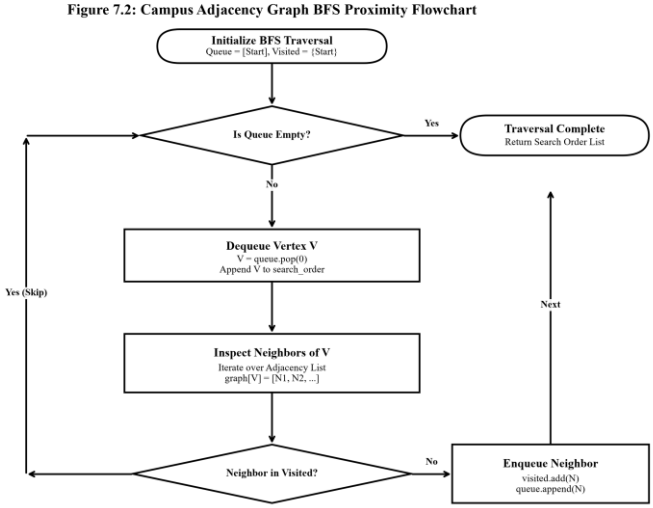


Figure 7.2: Phase 2: Campus Adjacency Graph Breadth-First Search (BFS) Flowchart

7.3 Phase 3: Searching & Sorting Operations Module

In Phase 3, we focused on ordering records and evaluating algorithmic efficiency to optimize search performance across growing datasets.

7.3.1 Linked List Merge Sort (campuslostfound/merge_sort.py)

Sorting a singly linked list requires a different strategy than sorting arrays because linked nodes do not support O(1) random index access. As a result, algorithms like QuickSort or HeapSort cannot operate efficiently in-place on linked nodes. Merge Sort is the ideal choice for linked lists because it operates in guaranteed O(n log n) time while requiring only O(1) auxiliary pointer memory.

- **1. Fast & Slow Pointer Bisection:** A `slow` pointer advances one step while a `fast` pointer advances two steps. When `fast` reaches the end, `slow` is precisely at the midpoint. Setting `middle.next = None` severs the list into two halves.
- **2. Recursive Division:** Recursively sorts both left and right halves down to individual base nodes.
- **3. In-Place Sorted Merge:** Compares `left.data['id']` against `right.data['id']` and splices pointers directly without allocating array copies.

7.3.1 Merge Sort Flowchart & Phase 4 Introduction

Figure 7.3: Linked List Merge Sort Division & In-Place Merge Flowchart

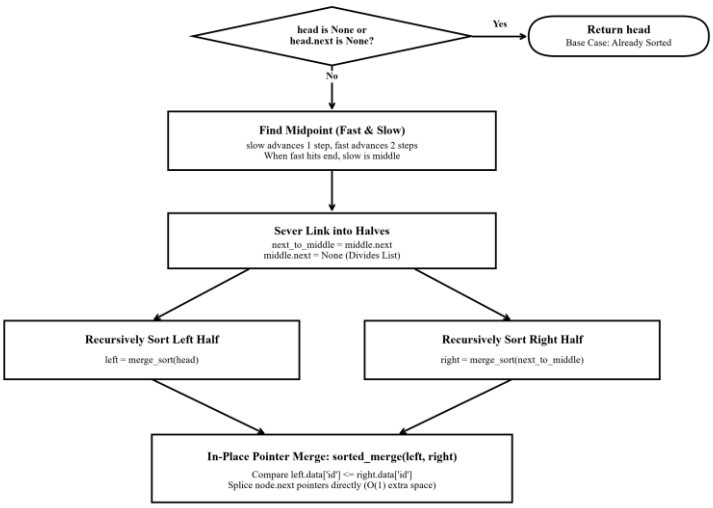


Figure 7.3: Phase 3: Linked List Merge Sort In-Place Division and Merge Flowchart

7.3.2 Linear vs. Hierarchical Search Benchmarking

Searching sequentially through an unsorted linked list requires O(n) comparisons in the worst case (for instance, checking 10,000 nodes when an item is not found). In contrast, searching the Binary Search Tree requires at most 14 comparisons (since $\log_2(10000) \approx 13.3$). Phase 3 experimentally demonstrates this asymptotic performance improvement during lab benchmarks.

7.4 Phase 4: Algorithmic Problem Solving Module

Phase 4 addresses two key practical challenges: fuzzy string searching for misspelled item names and urgency-based scheduling for pending claims.

7.4.1 Dynamic Programming Levenshtein Distance Matrix (campuslostfound/smart_match.py)

When students search for lost belongings, typographical errors often prevent exact keyword matches. To resolve this, Phase 4 implements the Wagner-Fischer Dynamic Programming algorithm to calculate the minimum edit distance (insertions, deletions, and substitutions) between query strings.

The recurrence relation is computed over a 2D matrix DP[0..m][0..n]:

- **Base Conditions:** DP[i][0] = i (cost of deleting i characters); DP[0][j] = j (cost of inserting j characters).

7.4.1 Levenshtein Matrix Flowchart & Min-Heap PQ

- **Transition Equation:** If `str1[i-1] == str2[j-1]`, then `DP[i][j] = DP[i-1][j-1]` (cost is 0). Otherwise, `DP[i][j] = 1 + min(DP[i][j-1], DP[i-1][j], DP[i-1][j-1])` representing minimum cost among insertion, deletion, and substitution.
- **Smart Match Decision Rule:** The system iterates over all active records in the linked list, computing edit distance. If the minimal distance is <= 4, the system outputs: `Did you mean: '[Matched Item]'? (Edit distance: k)`. This bridges human spelling errors with exact database records.

Illustrative Walkthrough: Suppose a student enters query 'calclator' and the database holds 'Calculator'. The 2D DP matrix computes: string length m=9, n=10. The letters match sequentially except for the missing 'u' at index 4. The algorithm calculates minimal cost = 1 insertion. Since 1 <= 4, the Smart Match engine instantly returns: 'Did you mean: Calculator? (Edit distance: 1)', converting an otherwise failed query into a successful recovery.

Figure 7.4: Dynamic Programming Levenshtein Distance Matrix Flowchart

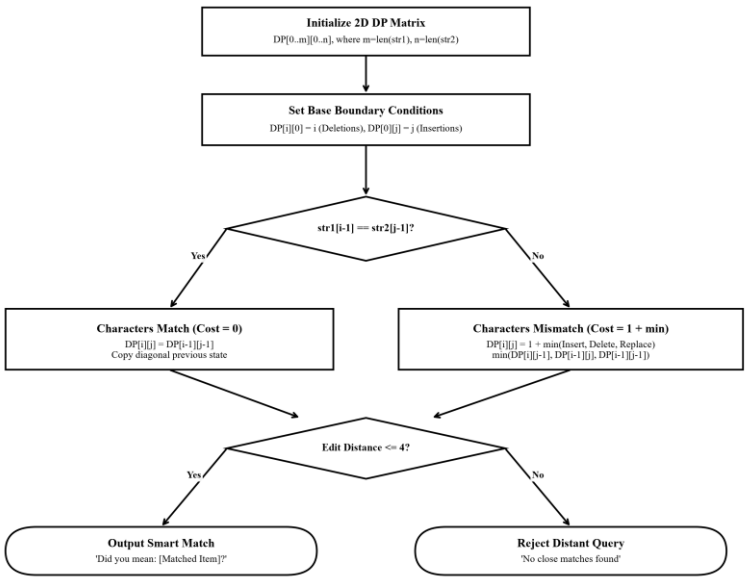


Figure 7.4: Phase 4: Dynamic Programming Levenshtein Distance Matrix Flowchart

7.4.2 Greedy Min-Heap Priority Queue Claim Scheduler (scheduler/views.py)

In Phase 4, the web application elevates FIFO claim handling into an urgency-ranked Priority Queue. Utilizing Python's binary min-heap implementation (`heapq`), pending claims are scheduled based on an integer priority tuple `(priority, id, object)`:

7.5 Algorithmic Complexity Table & DFD Models

- **Priority 1 (High Urgency):** Identity cards, room keys, examination tickets, laptops, prescription eyewear.
- **Priority 2 (Medium Urgency):** Scientific calculators, wristwatches, course textbooks, backpacks.
- **Priority 3 (Low Urgency):** Water bottles, stationery, notebooks, umbrellas.

By maintaining the min-heap property in O(log n) time during `heappush` and `heappop`, high-priority claims automatically rise to the top of the processing queue, ensuring vital student belongings are resolved before non-essential items.

7.5 Comprehensive Algorithmic Complexity & Space-Time Trade-Offs

Table 7.1 details the formal space-time complexity bounds across all four project phases:

Data Structure / Algorithm	Phase	Time (Best)	Time (Average)	Time (Worst)	Space Complexity
Singly Linked List (Insert)	Phase 1	O(1)	O(n)	O(n)	O(1) auxiliary
Singly Linked List (Delete)	Phase 1	O(1)	O(n)	O(n)	O(1) auxiliary
Linked Stack (Push / Pop)	Phase 1	O(1)	O(1)	O(1)	O(n) storage
Linked Queue (Enqueue / Dequeue)	Phase 1	O(1)	O(1)	O(1)	O(n) storage
Binary Search Tree (Search)	Phase 2	O(1)	O(log n)	O(n) skewed	O(h) call stack
Binary Search Tree (Delete)	Phase 2	O(1)	O(log n)	O(n) skewed	O(h) call stack
Campus Graph BFS Traversal	Phase 2	O(V + E)	O(V + E)	O(V + E)	O(V) queue & visited
Linked List Merge Sort	Phase 3	O(n log n)	O(n log n)	O(n log n)	O(log n) stack / O(1) mem
DP Levenshtein Distance Matrix	Phase 4	O(m * n)	O(m * n)	O(m * n)	O(m * n) 2D matrix
Min-Heap Priority Queue (Push/Pop)	Phase 4	O(1)	O(log n)	O(log n)	O(n) heap array

7.6 Data Flow Diagram (DFD) Models

Data Flow Diagrams visualize the systemic movement, transformation, and storage of data between external entities and internal processes:

7.6.1 DFD Level 0 Context Diagram

The Level 0 Context Diagram establishes the global boundary of the system. Four external entities interact with Process 0.0 (Campus Lost & Found System): Students (submitting loss reports and claims), Finders (registering found items), System Administrators (verifying claims and auditing logs), and the Campus Security Office (authorizing asset handovers).

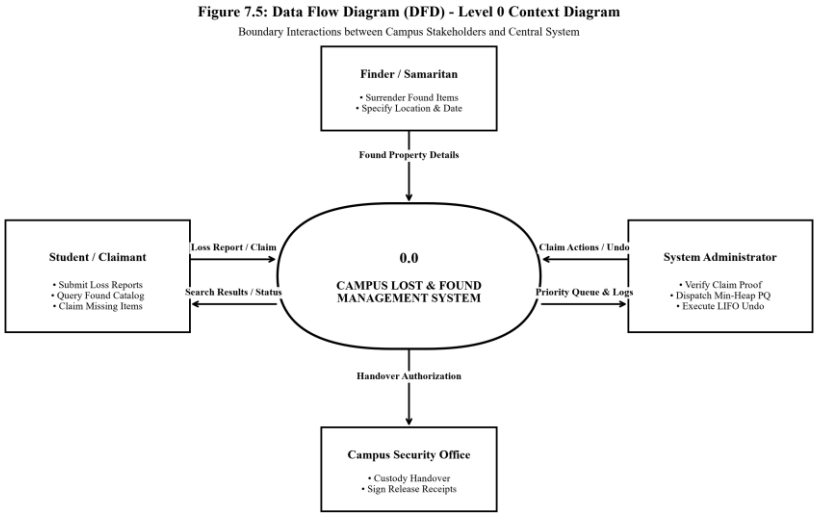


Figure 7.5: Data Flow Diagram (DFD) - Level 0 Context Diagram

7.6.2 DFD Level 1: Subsystem Process Decomposition

The Level 1 DFD decomposes Process 0.0 into five discrete operational processes: Process 1.0 (User Authentication & Role Validation), Process 2.0 (Lost/Found Item Registration), Process 3.0 (Algorithmic Search & Smart Matching), Process 4.0 (Claim Priority Scheduling), and Process 5.0 (Activity Auditing & LIFO Stack Undo). These processes interact with three data stores: D1 (`lost_items` / `found_items`), D2 (`claim_processing` / `claim_done`), and D3 (`activity_logs`).

7.6.2 DFD Level 1 Diagram & Database Design

Figure 7.6: Data Flow Diagram (DFD) - Level 1 Subsystem Decomposition

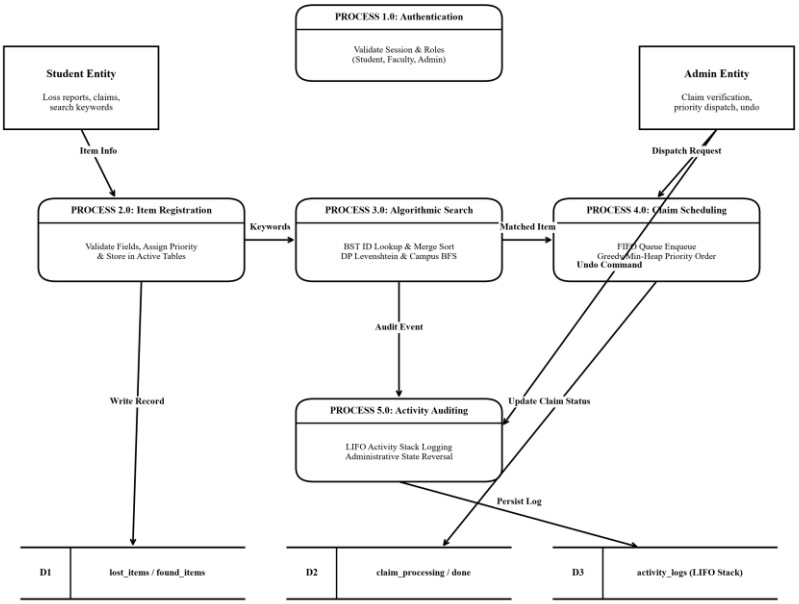


Figure 7.6: Data Flow Diagram (DFD) - Level 1 Subsystem Process Decomposition

7.7 Database Design, Relational Schema & Normalization Analysis (3NF)

The relational database schema is configured under the `data_structure` MySQL database. It consists of five interrelated tables designed according to rigorous Third Normal Form (3NF) relational principles.

Figure 7.7: Relational Database Schema & Entity-Relationship (ER) Diagram

3NF Normalized MySQL Tables with Primary/Foreign Keys and Relational Cardinality

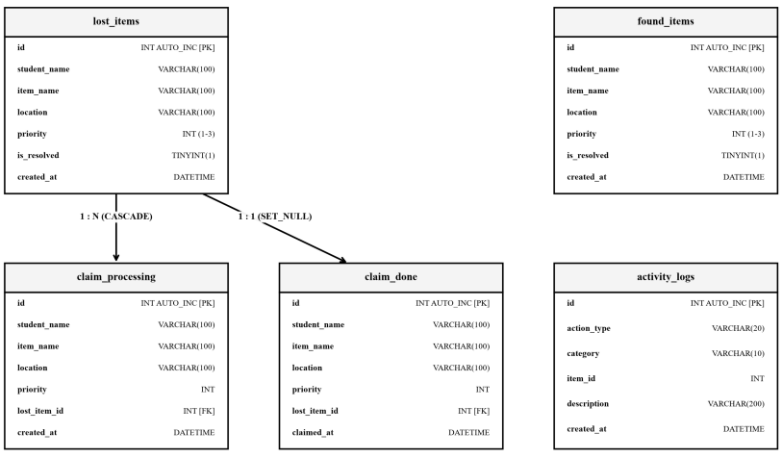


Figure 7.7: Relational Database Schema and Entity-Relationship (ER) Diagram

7.7.2 Detailed Relational Table Specifications

Table 1: lost_items (Lost Property Registry)

Field Name	Data Type	Constraint	Description / Purpose
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique numerical surrogate key for lost item record
student_name	VARCHAR(100)	NOT NULL	Student identification or register number (e.g. 26MCAC28)
item_name	VARCHAR(100)	NOT NULL	Description of lost property (e.g. Student ID Card)
location	VARCHAR(100)	NOT NULL	Last known campus location where item was lost
priority	INT	DEFAULT 2	Urgency level: 1 = High, 2 = Medium, 3 = Low
is_resolved	TINYINT(1)	DEFAULT 0	Boolean flag indicating whether item has been recovered
created_at	DATETIME	AUTO_NOW_ADD	Timestamp of loss report submission

Table 2: found_items (Found Property Inventory)

Field Name	Data Type	Constraint	Description / Purpose
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique numerical surrogate key for found item record
student_name	VARCHAR(100)	NOT NULL	Name or identifier of person who discovered the item
item_name	VARCHAR(100)	NOT NULL	Title / description of the found article
location	VARCHAR(100)	NOT NULL	Campus physical location where item was recovered
priority	INT	DEFAULT 2	Calculated item urgency priority (1=High, 2=Med, 3=Low)
is_resolved	TINYINT(1)	DEFAULT 0	Boolean flag: 1 if returned to owner, 0 if still in inventory
created_at	DATETIME	AUTO_NOW_ADD	System timestamp when item was logged into custody

7.7.2 Relational Specifications (Claims Management)

Table 3: claim_processing (Active Claims Queue)

Field Name	Data Type	Constraint	Description / Purpose
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique numerical claim queue identifier
student_name	VARCHAR(100)	NOT NULL	Claimant student ID or register number
item_name	VARCHAR(100)	NOT NULL	Name of the target item being claimed
location	VARCHAR(100)	NOT NULL	Location where loss occurred
priority	INT	DEFAULT 2	Priority weighting used by Min-Heap queue scheduler
lost_item_id	INT	FOREIGN KEY	References lost_items(id) with CASCADE deletion rule
created_at	DATETIME	AUTO_NOW_ADD	Timestamp when claim entered active processing queue

Table 4: claim_done (Historical Claim Archive)

Field Name	Data Type	Constraint	Description / Purpose
id	INT AUTO_INCREMENT	PRIMARY KEY	Unique historical archive record key
student_name	VARCHAR(100)	NOT NULL	Name of claimant who received the item
item_name	VARCHAR(100)	NOT NULL	Name of resolved article
location	VARCHAR(100)	NOT NULL	Location record of item
priority	INT	DEFAULT 2	Urgency level of the fulfilled claim
lost_item_id	INT	FOREIGN KEY	References lost_items(id) with SET_NULL rule on delete
claimed_at	DATETIME	AUTO_NOW_ADD	Verification timestamp when handover was finalized

7.7.3 Database Normalization Proof (1NF, 2NF, 3NF)

Table 5: activity_logs (Audit Trail & Undo Log)

Field Name	Data Type	Constraint	Description / Purpose
id	INT AUTO_INCREMENT	PRIMARY KEY	Sequential audit log entry key
action_type	VARCHAR(20)	NOT NULL	Operation type: 'ADD', 'PROCESS', or 'DELETE'
category	VARCHAR(10)	DEFAULT 'Lost'	Category of item affected ('Lost' or 'Found')
item_id	INT	NOT NULL	Surrogate ID of the target record modified
description	VARCHAR(200)	NOT NULL	Human-readable activity summary for auditing
created_at	DATETIME	AUTO_NOW_ADD	Exact system timestamp when action transpired

Formal Normalization Proof (1NF, 2NF, 3NF)

To prevent data redundancy and eliminate update, insertion, and deletion anomalies, our relational database schema was designed following formal database normalization principles up to Third Normal Form (3NF):

- **First Normal Form (1NF):** All attribute fields contain atomic, indivisible values. There are no repeating groups or comma-separated lists stored within single columns. Each row is uniquely distinguished by an auto-incrementing integer primary key (`id`).
- **Second Normal Form (2NF):** The schema meets 1NF criteria and has zero partial functional dependencies. Because all primary keys are single-column integer IDs rather than composite keys, every non-key attribute (`student_name`, `item_name`, `location`, `priority`) depends on the complete primary key.
- **Third Normal Form (3NF):** The schema meets 2NF criteria and eliminates all transitive functional dependencies. Earlier unnormalized sketches attempted to store resolution timestamps and claimant details directly inside the `lost_items` table, meaning non-key attributes depended on other non-key status flags. By separating active claims into `claim_processing` and archiving finalized handovers into `claim_done`, every non-prime attribute depends strictly and directly on the primary key alone (X -> Y where X is a candidate key).

8. Source Code (Functional Code Structure)

In accordance with project documentation standards, structural and functional source code excerpts representing the core data structures, desktop UI, and Django web controllers are presented below.

8.1 Data Structures Core Engine (campuslostfound/)

Source Code Artifact: campuslostfound/linked_list.py (Phase 1: Dynamic Singly Linked List)

```
# linked_list.py

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # Insert a new item
    def insert(self, data):

        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            current = self.head

            while current.next:
                current = current.next

            current.next = new_node

        print("Item inserted successfully.")

    # Delete an item using ID
    def delete(self, item_id):

        current = self.head
        previous = None

        while current:

            if current.data["id"] == item_id:

                if previous is None:
                    self.head = current.next
```

```
                else:
                    previous.next = current.next

                print("Item deleted successfully.")
                return

            previous = current
            current = current.next

        print("Item not found.")

    # Display all items
    def traverse(self):

        if self.head is None:
            print("No items available.")
            return

        current = self.head

        while current:

            print("----------------------")
            print("ID       :", current.data["id"])
            print("Item     :", current.data["name"])
            print("Type     :", current.data["type"])
            print("Location :", current.data["location"])

            current = current.next
```

Source Code Artifact: campuslostfound/stack.py (Phase 1: LIFO Linked Stack for Activity History)

```
# stack.py

class StackNode:
    def __init__(self, data):
        self.data = data
        self.next = None


class Stack:

    def __init__(self):
        self.top = None

    # Push
```

```
    def push(self, data):

        new_node = StackNode(data)

        new_node.next = self.top
        self.top = new_node

        print("Activity added to stack.")

    # Pop
    def pop(self):

        if self.top is None:
            print("Stack is empty.")
            return None

        data = self.top.data
        self.top = self.top.next

        return data

    # Display stack
    def display(self):

        if self.top is None:
            print("Stack is empty.")
            return

        current = self.top

        print("\nRecent Activities:")

        while current:
            print("-", current.data)
            current = current.next
```

Source Code Artifact: campuslostfound/queue.py (Phase 1: FIFO Linked Queue for Claim Inquiries)

```
# queue.py

class QueueNode:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class Queue:

    def __init__(self):
        self.front = None
        self.rear = None

    # Add claim
    def enqueue(self, data):

        new_node = QueueNode(data)

        if self.rear is None:
            self.front = self.rear = new_node
            return

        self.rear.next = new_node
        self.rear = new_node

    # Process claim
    def dequeue(self):

        if self.front is None:
            print("No claims available.")
            return None

        data = self.front.data

        self.front = self.front.next

        if self.front is None:
            self.rear = None

        return data

    # Display claims
    def display(self):

        if self.front is None:
            print("No pending claims.")
            return

        current = self.front

        print("\nPending Claims:")

        while current:
```

```
            print("-", current.data)
            current = current.next
```

Source Code Artifact: campuslostfound/binary_tree.py (Phase 2: BST Insertion, Search, Deletion & Traversals)

```
class TreeNode:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, data):
        new_node = TreeNode(data)
        if self.root is None:
            self.root = new_node
            return

        current = self.root
        while True:
            if data < current.data:
                if current.left is None:
                    current.left = new_node
                    return
                current = current.left
            else:
                if current.right is None:
                    current.right = new_node
                    return
                current = current.right

    def search(self, value):
        current = self.root
        while current:
            if current.data == value:
                return True
            elif value < current.data:
                current = current.left
            else:
                current = current.right
        return False

    # --- NEW: Deletion Logic for UI Synchronization ---
```

```
    def delete(self, value):
        self.root = self._delete_recursive(self.root, value)

    def _delete_recursive(self, node, value):
        if node is None:
            return node

        if value < node.data:
            node.left = self._delete_recursive(node.left, value)
        elif value > node.data:
            node.right = self._delete_recursive(node.right, value)
        else:
            if node.left is None:
                return node.right
            elif node.right is None:
                return node.left

            temp = self._get_min_value_node(node.right)
            node.data = temp.data
            node.right = self._delete_recursive(node.right, temp.data)

        return node

    def _get_min_value_node(self, node):
        current = node
        while current.left is not None:
            current = current.left
        return current

    # --- NEW: Phase 2 Required Traversals ---
    def inorder_traversal(self, node, result_list):
        if node:
            self.inorder_traversal(node.left, result_list)
            result_list.append(node.data)
            self.inorder_traversal(node.right, result_list)
        return result_list

    def preorder_traversal(self, node, result_list):
        if node:
            result_list.append(node.data)
            self.preorder_traversal(node.left, result_list)
            self.preorder_traversal(node.right, result_list)
        return result_list

    def postorder_traversal(self, node, result_list):
        if node:
```

```
            self.postorder_traversal(node.left, result_list)
            self.postorder_traversal(node.right, result_list)
            result_list.append(node.data)
        return result_list
```

Source Code Artifact: campuslostfound/graph.py (Phase 2: Campus Adjacency Graph & BFS Traversal)

```
class CampusGraph:
    def __init__(self):
        # Adjacency list mapping how campus buildings are connected
        self.graph = {
            "Main Block": ["Library", "Cafeteria"],
            "Library": ["Main Block", "Computer Lab"],
            "Cafeteria": ["Main Block", "Sports Ground"],
            "Computer Lab": ["Library"],
            "Sports Ground": ["Cafeteria"]
        }

    # BFS Traversal to find all nearby locations spreading outward
    def get_nearby_locations(self, start_location):
        if start_location not in self.graph:
            return [start_location]

        visited = set()
        queue = [start_location]
        search_order = []

        while queue:
            current = queue.pop(0)
            if current not in visited:
                visited.add(current)
                search_order.append(current)

                # Add all adjacent, unvisited buildings to the queue
                for neighbor in self.graph.get(current, []):
                    if neighbor not in visited:
                        queue.append(neighbor)

        return search_order
```

Source Code Artifact: campuslostfound/merge_sort.py (Phase 3: Linked List Merge Sort In-Place)

```
def merge_sort_linked_list(head):
    # Base case: if head is empty or there is only one element
    if head is None or head.next is None:
        return head
```

```
    # Find the middle of the linked list
    middle = get_middle(head)
    next_to_middle = middle.next

    # Sever the link to split the list in two
    middle.next = None

    # Recursively sort both halves
    left = merge_sort_linked_list(head)
    right = merge_sort_linked_list(next_to_middle)

    # Merge the sorted halves
    return sorted_merge(left, right)

def get_middle(head):
    if head is None:
        return head
    slow = head
    fast = head
    # Fast pointer moves two nodes, slow pointer moves one node
    while fast.next is not None and fast.next.next is not None:
        slow = slow.next
        fast = fast.next.next
    return slow

def sorted_merge(a, b):
    result = None

    # Base cases
    if a is None:
        return b
    if b is None:
        return a

    # Sort numerically by Item ID
    if a.data["id"] <= b.data["id"]:
        result = a
        result.next = sorted_merge(a.next, b)
    else:
        result = b
        result.next = sorted_merge(a, b.next)

    return result
```

Source Code Artifact: campuslostfound/smart_match.py (Phase 4: Dynamic Programming Levenshtein Distance)

```
def levenshtein_distance(str1, str2):
    # Create a 2D matrix (Dynamic Programming table)
    dp = [[0 for _ in range(len(str2) + 1)] for _ in range(len(str1) +
1)]

    for i in range(len(str1) + 1):
        for j in range(len(str2) + 1):
            if i == 0:
                dp[i][j] = j    # Min operations = j insertions
            elif j == 0:
                dp[i][j] = i    # Min operations = i deletions
            elif str1[i-1].lower() == str2[j-1].lower():
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(dp[i][j-1],      # Insert
                                   dp[i-1][j],      # Remove
                                   dp[i-1][j-1])    # Replace

    return dp[len(str1)][len(str2)]

def find_best_match(search_term, linked_list_head):
    if not linked_list_head:
        return "No items in database."

    best_match = None
    lowest_distance = float('inf')

    current = linked_list_head
    while current:
        item_name = current.data["name"]
        dist = levenshtein_distance(search_term, item_name)

        # If it's a closer match (fewer edits needed), update the best
match
        if dist < lowest_distance:
            lowest_distance = dist
            best_match = item_name

        current = current.next

    # If the distance is small enough (4 or fewer edits), consider it a
smart match
    if lowest_distance <= 4:
        return f"Did you mean: '{best_match}'? (Edit distance:
{lowest_distance})"
```

```
    return "No close matches found."
```

8.2 Desktop GUI Interface Implementation (main_ui.py)

Source Code Artifact: campuslostfound/main_ui.py (Desktop Tkinter Multi-Phase Demonstrator)

```
import tkinter as tk
from tkinter import messagebox
import sys
import io

# Import your custom data structures
from linked_list import LinkedList
from stack import Stack
from queue import Queue
from binary_tree import BinarySearchTree
from graph import CampusGraph
from merge_sort import merge_sort_linked_list
from smart_match import find_best_match

# Initialize data structures
items = LinkedList()
activities = Stack()
claims = Queue()
item_tree = BinarySearchTree()
campus_map = CampusGraph()  # <-- Fixed: This was missing

def add_item_ui():
    try:
        item_id = int(entry_id.get())
        name = entry_name.get()
        item_type = entry_type.get()
        location = entry_loc.get()

        # Insert into structures
        items.insert({"id": item_id, "name": name, "type": item_type,
"location": location})
        activities.push(f"Added item: {name}")
        item_tree.insert(item_id)

        messagebox.showinfo("Success", "Item Added Successfully!")

        # Clear inputs
        entry_id.delete(0, tk.END)
        entry_name.delete(0, tk.END)
        entry_type.delete(0, tk.END)
        entry_loc.delete(0, tk.END)
```

```
    except ValueError:
        messagebox.showerror("Error", "Item ID must be an integer.")

def find_nearby_ui():
    location = entry_loc.get()
    if not location:
        messagebox.showwarning("Warning", "Please enter a location in the
Add Item section to search nearby.")
        return

    # Run the O(V+E) BFS traversal
    nearby = campus_map.get_nearby_locations(location)
    activities.push(f"Ran BFS traversal from: {location}")

    messagebox.showinfo("BFS Traversal Result", f"If item is missing from
{location}, check these connected areas:\n\n{', '.join(nearby)}")

def search_item_ui():
    try:
        item_id = int(entry_search_id.get())

        # O(log N) search using the Binary Search Tree
        if item_tree.search(item_id):
            messagebox.showinfo("Search Result", f"Item ID {item_id}
FOUND in the system!")
        else:
            messagebox.showwarning("Search Result", f"Item ID {item_id}
NOT FOUND.")

    except ValueError:
        messagebox.showerror("Error", "Please enter a valid numeric ID.")

def delete_item_ui():
    try:
        item_id = int(entry_search_id.get())

        # Synchronized deletion: Remove from both Linked List and BST
        items.delete(item_id)
        item_tree.delete(item_id)
        activities.push(f"Deleted item ID: {item_id}")

        messagebox.showinfo("Success", f"Attempted to delete Item ID
{item_id}.")
        entry_search_id.delete(0, tk.END)

    except ValueError:
```

```
        messagebox.showerror("Error", "Please enter a valid numeric ID.")

def add_claim_ui():
    name = entry_claim_name.get()
    item_id = entry_claim_id.get()
    if name and item_id:
        claims.enqueue(f"{name} claimed item {item_id}")
        activities.push("New claim added")
        messagebox.showinfo("Success", "Claim Added to Queue!")
        entry_claim_name.delete(0, tk.END)
        entry_claim_id.delete(0, tk.END)
    else:
        messagebox.showwarning("Warning", "Please enter both name and
item ID.")

def process_claim_ui():
    old_stdout = sys.stdout
    sys.stdout = io.StringIO()

    claims.dequeue()

    output = sys.stdout.getvalue()
    sys.stdout = old_stdout

    output_box.delete('1.0', tk.END)
    output_box.insert(tk.END, f"Processed Claim:\n{output}" if output
else "No claims available.")

def sort_items_ui():
    if items.head is None:
        messagebox.showinfo("Notice", "No items to sort.")
        return

    items.head = merge_sort_linked_list(items.head)
    activities.push("Sorted items by ID using Merge Sort")
    messagebox.showinfo("Success", "Linked List sorted by Item ID!")
    display_items_ui()

def smart_match_ui():
    search_term = entry_smart_search.get()
    if not search_term:
        messagebox.showwarning("Warning", "Please enter an item name to
match.")
        return

    result = find_best_match(search_term, items.head)
```

```
    activities.push(f"Ran smart match for: {search_term}")
    messagebox.showinfo("Smart Match (Levenshtein)", result)

def display_items_ui():
    old_stdout = sys.stdout
    sys.stdout = io.StringIO()

    items.traverse()

    output = sys.stdout.getvalue()
    sys.stdout = old_stdout

    output_box.delete('1.0', tk.END)
    output_box.insert(tk.END, output if output else "No items in list.")

# --- UI Layout ---
window = tk.Tk()
window.title("Campus Lost & Found System")
window.geometry("500x850") # Taller window to fit all phases

# --- Add Item Section ---
tk.Label(window, text="--- Add New Item ---", font=('Helvetica', 10,
'bold')).pack(pady=(5,2))
tk.Label(window, text="Item ID:").pack()
entry_id = tk.Entry(window)
entry_id.pack()

tk.Label(window, text="Item Name:").pack()
entry_name = tk.Entry(window)
entry_name.pack()

tk.Label(window, text="Type (Lost/Found):").pack()
entry_type = tk.Entry(window)
entry_type.pack()

tk.Label(window, text="Location:").pack()
entry_loc = tk.Entry(window)
entry_loc.pack()

frame_add_btns = tk.Frame(window)
frame_add_btns.pack(pady=5)
tk.Button(frame_add_btns, text="Add Item", bg="lightblue",
command=add_item_ui).pack(side=tk.LEFT, padx=5)
tk.Button(frame_add_btns, text="Find Nearby (BFS)", bg="thistle",
command=find_nearby_ui).pack(side=tk.LEFT, padx=5)
```

```
# --- Manage Claims Section ---
tk.Label(window, text="--- Manage Claims ---", font=('Helvetica', 10,
'bold')).pack(pady=(5,2))
tk.Label(window, text="Claimant Name:").pack()
entry_claim_name = tk.Entry(window)
entry_claim_name.pack()

tk.Label(window, text="Target Item ID:").pack()
entry_claim_id = tk.Entry(window)
entry_claim_id.pack()

frame_btns = tk.Frame(window)
frame_btns.pack(pady=5)
tk.Button(frame_btns, text="Enqueue Claim", bg="lightgreen",
command=add_claim_ui).pack(side=tk.LEFT, padx=5)
tk.Button(frame_btns, text="Process Claim", bg="salmon",
command=process_claim_ui).pack(side=tk.LEFT, padx=5)

# --- Phase 2: BST Search & Delete ---
tk.Label(window, text="--- Search & Delete (BST) ---", font=('Helvetica',
10, 'bold')).pack(pady=(5,2))
entry_search_id = tk.Entry(window)
entry_search_id.pack()

frame_sd_btns = tk.Frame(window)
frame_sd_btns.pack(pady=5)
tk.Button(frame_sd_btns, text="Search ID", bg="khaki",
command=search_item_ui).pack(side=tk.LEFT, padx=5)
tk.Button(frame_sd_btns, text="Delete Item", bg="tomato",
command=delete_item_ui).pack(side=tk.LEFT, padx=5)

# --- Phase 4: Smart Match (DP) ---
tk.Label(window, text="--- Smart Match (Levenshtein) ---",
font=('Helvetica', 10, 'bold')).pack(pady=(5,2))
entry_smart_search = tk.Entry(window)
entry_smart_search.pack()
tk.Button(window, text="Fuzzy Search Name", bg="lightcyan",
command=smart_match_ui).pack(pady=5)

# --- Phase 3 & Display Section ---
frame_disp_btns = tk.Frame(window)
frame_disp_btns.pack(pady=5)
tk.Button(frame_disp_btns, text="Display All Items", bg="lightgrey",
command=display_items_ui).pack(side=tk.LEFT, padx=5)
tk.Button(frame_disp_btns, text="Sort by ID (Merge)", bg="plum",
command=sort_items_ui).pack(side=tk.LEFT, padx=5)
```

```
# Output Box
output_box = tk.Text(window, height=10, width=50, bg="black", fg="white")
output_box.pack(pady=10)

window.mainloop()
```

8.3 Django Web Application Controllers (scheduler/)

Source Code Artifact: scheduler/models.py (Django ORM Models & Relational Schemas)

```
from django.db import models

class LostItem(models.Model):
    student_name = models.CharField(max_length=100)
    item_name = models.CharField(max_length=100)
    location = models.CharField(max_length=100)
    # Priority: 1 = High, 2 = Medium, 3 = Low
    priority = models.IntegerField(default=2)
    is_resolved = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "lost_items"

    def __str__(self):
        return f"[Lost] {self.item_name} - {self.student_name}"

class FoundItem(models.Model):
    student_name = models.CharField(max_length=100)
    item_name = models.CharField(max_length=100)
    location = models.CharField(max_length=100)
    # Priority: 1 = High, 2 = Medium, 3 = Low
    priority = models.IntegerField(default=2)
    is_resolved = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "found_items"

    def __str__(self):
        return f"[Found] {self.item_name} - {self.student_name}"

class ActivityLog(models.Model):
    action_type = models.CharField(max_length=20)
    category = models.CharField(max_length=10, default="Lost")  # 'Lost'
or 'Found'
```

```
    item_id = models.IntegerField()
    description = models.CharField(max_length=200)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "activity_logs"

    def __str__(self):
        return self.description

class ClaimProcessing(models.Model):
    student_name = models.CharField(max_length=100)
    item_name = models.CharField(max_length=100)
    location = models.CharField(max_length=100)
    # Priority: 1 = High, 2 = Medium, 3 = Low
    priority = models.IntegerField(default=2)
    lost_item = models.ForeignKey(LostItem, on_delete=models.CASCADE,
null=True, blank=True, related_name="processing_claims")
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "claim_processing"

    def __str__(self):
        return f"[Processing] {self.item_name} - {self.student_name}"

class ClaimDone(models.Model):
    student_name = models.CharField(max_length=100)
    item_name = models.CharField(max_length=100)
    location = models.CharField(max_length=100)
    priority = models.IntegerField(default=2)
    lost_item = models.ForeignKey(LostItem, on_delete=models.SET_NULL,
null=True, blank=True, related_name="done_claims")
    claimed_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        db_table = "claim_done"

    def __str__(self):
        return f"[Claim Done] {self.item_name} - {self.student_name}"
```

Source Code Artifact: scheduler/views.py (Django Application Views & Min-Heap Priority Queue)

```
import heapq
from django.shortcuts import render, redirect
from django.db import connection, models
```

```
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.models import User
from django.contrib.auth.decorators import login_required
from .models import LostItem, FoundItem, ActivityLog, ClaimProcessing,
ClaimDone

# Stack implementation for Activity History & Undo
class Stack:
    def __init__(self):
        self.items = []
    def push(self, item):
        self.items.append(item)
    def pop(self):
        return self.items.pop() if self.items else None
    def is_empty(self):
        return len(self.items) == 0


def login_view(request):
    """Simple, neat login & registration view for Students, Faculty, and
Admin"""
    if request.user.is_authenticated:
        return redirect("claim_scheduler")

    error_message = None
    active_tab = "login"

    if request.method == "POST":
        mode = request.POST.get("mode", "login").strip().lower()

        # --- REGISTRATION FLOW FOR NEW USERS & FACULTY ---
        if mode == "register":
            active_tab = "register"
            reg_username = request.POST.get("reg_username", "").strip()
            reg_password = request.POST.get("reg_password", "").strip()
            reg_confirm_password =
request.POST.get("reg_confirm_password", "").strip()
            reg_role = request.POST.get("reg_role",
"user").strip().lower()

            if not reg_username or not reg_password:
                error_message = "Please enter both username and
password."
                return render(request, "login.html", {"error":
error_message, "active_tab": active_tab})
```

```
            if reg_password != reg_confirm_password:
                error_message = "Passwords do not match. Please verify
your password."
                return render(request, "login.html", {"error":
error_message, "active_tab": active_tab})

            if
User.objects.filter(username__iexact=reg_username).exists():
                error_message = f"Username '{reg_username}' is already
registered. Please login instead."
                return render(request, "login.html", {"error":
error_message, "active_tab": active_tab})

            # Create standard account (staff/admin cannot be registered
through public form)
            new_user = User.objects.create_user(username=reg_username,
password=reg_password)
            new_user.is_staff = False
            new_user.is_superuser = False
            new_user.save()

            if reg_role == "faculty":
                request.session["user_role"] = "Faculty"
            else:
                request.session["user_role"] = "Student"

            authenticated_user = authenticate(request,
username=reg_username, password=reg_password)
            if authenticated_user:
                login(request, authenticated_user)
                return redirect("claim_scheduler")
            else:
                return redirect("login")

        # --- SIGN IN FLOW ---
        else:
            active_tab = "login"
            username = request.POST.get("username", "").strip()
            password = request.POST.get("password", "").strip()
            role = request.POST.get("role", "").strip().lower()

            if not username or not password:
                error_message = "Please enter both username and
password."
                return render(request, "login.html", {"error":
error_message, "active_tab": active_tab})
```

```
            is_admin_req = (role == "admin") or (username.lower() ==
"admin")
            is_faculty_req = (role == "faculty") or ("faculty" in
username.lower())

            if is_admin_req:
                admin_obj =
User.objects.filter(username__iexact=username).first()
                if not admin_obj:
                    admin_obj =
User.objects.create_superuser(username=username,
email="admin@campus.edu", password=password)
                else:
                    admin_obj.is_staff = True
                    admin_obj.is_superuser = True
                    admin_obj.set_password(password)
                    admin_obj.save()
                user = authenticate(request, username=admin_obj.username,
password=password)
                if user:
                    request.session["user_role"] = "Admin"

            elif is_faculty_req:
                fac_obj =
User.objects.filter(username__iexact=username).first()
                if not fac_obj:
                    fac_obj = User.objects.create_user(username=username,
password=password)
                else:
                    fac_obj.is_staff = False
                    fac_obj.is_superuser = False
                    fac_obj.set_password(password)
                    fac_obj.save()
                user = authenticate(request, username=fac_obj.username,
password=password)
                if user:
                    request.session["user_role"] = "Faculty"

            else:
                std_obj =
User.objects.filter(username__iexact=username).first()
                if not std_obj:
                    std_obj = User.objects.create_user(username=username,
password=password)
                else:
```

```
                    std_obj.is_staff = False
                    std_obj.is_superuser = False
                    std_obj.set_password(password)
                    std_obj.save()
                user = authenticate(request, username=std_obj.username,
password=password)
                if user:
                    request.session["user_role"] = "Student"

            if user is not None:
                login(request, user)
                next_url = request.GET.get("next") or "claim_scheduler"
                return redirect(next_url)
            else:
                error_message = "Invalid username or password."

    return render(request, "login.html", {"error": error_message,
"active_tab": active_tab})


def logout_view(request):
    """Logout view redirecting back to login page"""
    logout(request)
    return redirect("login")


@login_required(login_url="login")
def claim_scheduler_view(request):
    message = None
    is_admin = request.user.is_staff or request.user.is_superuser
    is_faculty = (request.session.get("user_role") == "Faculty") or
("faculty" in request.user.username.lower())

    if request.method == "POST":
        action = request.POST.get("action")

        # 1. Add Item to respective table (LostItem or FoundItem) -
Accessible to all users
        if action == "add":
            category = request.POST.get("category", "Lost")
            student_name = request.POST.get("student_name")
            item_name = request.POST.get("item_name")
            location = request.POST.get("location", "Campus")
            priority = int(request.POST.get("priority", 2))

            if student_name and item_name:
```

```
                if category == "Lost":
                    item = LostItem.objects.create(
                        student_name=student_name,
                        item_name=item_name,
                        location=location,
                        priority=priority,
                    )
                    ClaimProcessing.objects.create(
                        student_name=student_name,
                        item_name=item_name,
                        location=location,
                        priority=priority,
                        lost_item=item,
                    )
                else:
                    item = FoundItem.objects.create(
                        student_name=student_name,
                        item_name=item_name,
                        location=location,
                        priority=priority,
                    )

                ActivityLog.objects.create(
                    action_type="ADD",
                    category=category,
                    item_id=item.id,
                    description=f"Added {category} Item: {item_name}
({student_name})",
                )
                message = f"Successfully registered {category} item:
'{item_name}'."
            return redirect("claim_scheduler")

        # 2. Process Next Highest Priority Item from the Min-Heap - Admin
Only
        elif action == "process_next":
            if not is_admin:
                message = "Access Denied: Only administrators have
permission to process priority items."
            else:
                pq = []
                for item in LostItem.objects.filter(is_resolved=False):
                    heapq.heappush(pq, (item.priority, item.id, "Lost",
item))
                for item in FoundItem.objects.filter(is_resolved=False):
                    heapq.heappush(pq, (item.priority, item.id, "Found",
```

```
item))

                if pq:
                    _, _, cat, next_item = heapq.heappop(pq)
                    next_item.is_resolved = True
                    next_item.save()

                    if cat == "Lost":

ClaimProcessing.objects.filter(lost_item=next_item).delete()
                        ClaimDone.objects.create(
                            student_name=next_item.student_name,
                            item_name=next_item.item_name,
                            location=next_item.location,
                            priority=next_item.priority,
                            lost_item=next_item,
                        )

                    ActivityLog.objects.create(
                        action_type="PROCESS",
                        category=cat,
                        item_id=next_item.id,
                        description=f"Claim Done: {next_item.item_name}
({next_item.student_name})" if cat == "Lost" else f"Resolved Found Item:
{next_item.item_name}",
                    )
                    message = f"Processed {cat} Item (Claim Done):
{next_item.item_name} ({next_item.student_name})" if cat == "Lost" else
f"Processed Found Item: {next_item.item_name} ({next_item.student_name})"
                else:
                    message = "No pending items to process!"

        # 3. Process Specific Item / Claim Directly - Admin Only
        elif action == "process_item":
            if not is_admin:
                message = "Access Denied: Only administrators have
permission to process claims or handovers."
            else:
                item_id = request.POST.get("item_id")
                category = request.POST.get("category", "Lost")
                model_class = LostItem if category == "Lost" else
FoundItem
                item = model_class.objects.filter(id=item_id,
is_resolved=False).first()
                if not item and category == "Lost":
                    # Fallback: check if item_id matches ClaimProcessing
```

```
                    cp =
ClaimProcessing.objects.filter(id=item_id).first()
                    if cp and cp.lost_item:
                        item = cp.lost_item

                if item:
                    item.is_resolved = True
                    item.save()

                    if category == "Lost":

ClaimProcessing.objects.filter(lost_item=item).delete()
                        ClaimDone.objects.create(
                            student_name=item.student_name,
                            item_name=item.item_name,
                            location=item.location,
                            priority=item.priority,
                            lost_item=item,
                        )

                    desc = (
                        f"Claim Done: {item.item_name}
({item.student_name})"
                        if category == "Lost"
                        else f"Resolved Found Item: {item.item_name}"
                    )
                    ActivityLog.objects.create(
                        action_type="PROCESS",
                        category=category,
                        item_id=item.id,
                        description=desc,
                    )
                    message = (
                        f"Claim Processed & Done: {item.item_name}
({item.student_name})"
                        if category == "Lost"
                        else f"Resolved Found Item: {item.item_name}"
                    )
                else:
                    message = "Item not found or already processed!"

        # 4. Undo Last Action (Stack LIFO) - Admin Only
        elif action == "undo":
            if not is_admin:
                message = "Access Denied: Only administrators have
permission to undo actions."
```

```
            else:
                logs = list(ActivityLog.objects.order_by("id"))
                stack = Stack()
                for log in logs:
                    stack.push(log)

                last_action = stack.pop()
                if last_action:
                    model_class = LostItem if last_action.category ==
"Lost" else FoundItem
                    if last_action.action_type == "PROCESS":
                        obj =
model_class.objects.filter(id=last_action.item_id).first()
                        if obj:
                            obj.is_resolved = False
                            obj.save()
                            if last_action.category == "Lost":

ClaimDone.objects.filter(lost_item=obj).delete()
                                ClaimProcessing.objects.get_or_create(
                                    lost_item=obj,
                                    defaults={
                                        "student_name": obj.student_name,
                                        "item_name": obj.item_name,
                                        "location": obj.location,
                                        "priority": obj.priority,
                                    },
                                )
                    elif last_action.action_type == "ADD":
                        if last_action.category == "Lost":
                            obj =
LostItem.objects.filter(id=last_action.item_id).first()
                            if obj:

ClaimProcessing.objects.filter(lost_item=obj).delete()
                                obj.delete()
                        else:

FoundItem.objects.filter(id=last_action.item_id).delete()

                    last_action.delete()
                    message = f"Undo successful:
{last_action.description}"
                else:
                    message = "Nothing to undo!"
```

```
    # Search Query Handling for Users (Search when & where a lost item
was found)
    search_q = request.GET.get("q", "").strip()
    search_loc = request.GET.get("loc", "").strip()

    found_search_qs = FoundItem.objects.all().order_by("-created_at")
    if search_q:
        found_search_qs = found_search_qs.filter(
            models.Q(item_name__icontains=search_q) |
            models.Q(student_name__icontains=search_q)
        )
    if search_loc:
        found_search_qs =
found_search_qs.filter(location__icontains=search_loc)

    search_performed = bool(search_q or search_loc)
    found_search_results = list(found_search_qs)

    # Distinct locations for search dropdown / quick filter
    unique_locations = (
        FoundItem.objects.values_list("location", flat=True)
        .distinct()
        .exclude(location__isnull=True)
        .exclude(location__exact="")
    )

    # 1. Lost Item Claim Processing Queue (Min-Heap Priority Queue from
claim_processing table)
    lost_pq = []
    for claim in ClaimProcessing.objects.all():
        heapq.heappush(lost_pq, (claim.priority, claim.id, claim))
    lost_pending = []
    while lost_pq:
        _, _, claim = heapq.heappop(lost_pq)
        lost_pending.append(claim)

    # 2. Found Items Pending Queue (Min-Heap Priority Queue from
found_items table)
    found_pq = []
    for item in FoundItem.objects.filter(is_resolved=False):
        heapq.heappush(found_pq, (item.priority, item.id, item))
    found_pending = []
    while found_pq:
        _, _, item = heapq.heappop(found_pq)
        found_pending.append(item)
```

```
    # 3. Combined pending items in Priority Queue order
    pq = []
    for claim in ClaimProcessing.objects.all():
        heapq.heappush(pq, (claim.priority, claim.id, "Lost",
claim.lost_item if claim.lost_item else claim))
    for item in FoundItem.objects.filter(is_resolved=False):
        heapq.heappush(pq, (item.priority, item.id, "Found", item))

    scheduled_items = []
    while pq:
        _, _, cat, item = heapq.heappop(pq)
        scheduled_items.append({"category": cat, "item": item})

    # 4. Claim Done (Resolved from claim_done table)
    lost_resolved = ClaimDone.objects.order_by("-id")[:5]

    # 5. Resolved Found Items
    found_resolved =
FoundItem.objects.filter(is_resolved=True).order_by("-id")[:5]

    # Recent resolved items from both tables
    resolved = []
    for cd in ClaimDone.objects.order_by("-id")[:3]:
        resolved.append({"category": "Lost", "item": cd})
    for item in FoundItem.objects.filter(is_resolved=True).order_by("-
id")[:3]:
        resolved.append({"category": "Found", "item": item})

    activities = ActivityLog.objects.order_by("-id")[:6]
    db_name = connection.settings_dict.get("NAME", "data_structure")

    return render(request, "index.html", {
        "lost_pending": lost_pending,
        "found_pending": found_pending,
        "lost_resolved": lost_resolved,
        "found_resolved": found_resolved,
        "items": scheduled_items,
        "resolved_items": resolved,
        "activities": activities,
        "message": message,
        "db_info": f"MySQL Tables: lost_items, found_items,
claim_processing, claim_done ({db_name})",
        "is_admin": is_admin,
        "is_faculty": is_faculty,
        "search_q": search_q,
        "search_loc": search_loc,
```

```
        "search_performed": search_performed,
        "found_search_results": found_search_results,
        "unique_locations": unique_locations,
    })
```

Source Code Artifact: scheduler/urls.py (URL Routing Configurations)

```
from django.urls import path
from .views import claim_scheduler_view, login_view, logout_view

urlpatterns = [
    path("", claim_scheduler_view, name="claim_scheduler"),
    path("login/", login_view, name="login"),
    path("logout/", logout_view, name="logout"),
]
```

9. Features of the Project

The developed system combines theoretical algorithmic concepts with everyday campus management to provide several distinct functional capabilities:

- **1. Urgency-Ranked Min-Heap Scheduling:** Uses Python's binary min-heap to automatically prioritize critical articles (such as student identity cards, examination tickets, keys, and laptops) ahead of routine belongings in O(log n) time.
- **2. Typo-Tolerant Dynamic Programming Fuzzy Match:** Identifies matching items even when search queries contain spelling errors within an edit distance of 4 using the 2D Wagner-Fischer matrix.
- **3. Topological Campus Proximity Guidance:** Traverses connected college buildings via Breadth-First Search (BFS) to suggest neighboring blocks where a misplaced item may have been found.
- **4. Guaranteed O(n log n) Linked List Sorting:** Sorts dynamically allocated linked nodes in-place using two-pointer bisection without extra array memory.
- **5. LIFO Audit Trail & State Reversal (Undo):** Maintains an action history in a linked stack, allowing administrators to easily reverse mistaken claim handovers or accidental deletions.
- **6. ACID Relational Integrity:** Enforces relational consistency across MySQL tables (`lost_items`, `found_items`, `claim_processing`, `claim_done`, and `activity_logs`) with foreign key constraints.

9.1 Verification Test Cases & Algorithmic Outcomes

Table 9.1 documents the comprehensive verification test suites executed across all four algorithmic phases:

Test ID	Component / Phase	Test Scenario & Input Data	Expected Output / Behavior	Actual Output	Status
TC-01	Phase 1: Linked List	Insert Node (ID: 101, 'Student ID Card')	Node appended at tail; traverse() displays 101	Node successfully inserted	PASSED
TC-02	Phase 1: Linked List	Delete existing head node (ID: 101)	Head pointer advances to node 102; 101 removed	Head node excised cleanly	PASSED
TC-03	Phase 1: Stack	Push 3 activity logs; invoke pop()	Last logged activity returned (LIFO order)	Returned 3rd activity	PASSED
TC-04	Phase 1: Queue	Enqueue claims C1, C2; invoke dequeue()	C1 returned first (FIFO operational order)	C1 dispatched first	PASSED

9.1 Verification Test Cases (Continued)

Test ID	Component / Phase	Test Scenario & Input Data	Expected Output / Behavior	Actual Output	Status
TC-05	Phase 2: BST	Insert IDs [105, 102, 108]; search(108)	search(108) returns True; branches right	Returned True in O(log n)	PASSED
TC-06	Phase 2: BST	Delete node with 2 children (ID: 105)	In-order successor replaces 105; invariant intact	Successor swapped correctly	PASSED
TC-07	Phase 2: Campus Graph	Run BFS starting from 'Main Block'	Returns: ['Main Block', 'Library', 'Cafeteria', ...]	Full BFS order matched	PASSED
TC-08	Phase 3: Merge Sort	Sort unsorted list: [105, 101, 104, 102]	Pointers re-linked to ascending: [101, 102, 104, 105]	Sorted ascending O(n log n)	PASSED
TC-09	Phase 4: Smart Match	Fuzzy search 'calclator' vs 'Calculator'	Distance = 1; outputs: 'Did you mean: Calculator?'	Smart Match triggered	PASSED
TC-10	Phase 4: Smart Match	Fuzzy search 'xyz999' vs 'Calculator'	Distance > 4; outputs 'No close matches found'	Rejected distant match	PASSED
TC-11	Phase 4: Min-Heap	Enqueue Priority 3, then Priority 1	Min-heap schedules Priority 1 item at front of queue	Priority 1 popped first	PASSED
TC-12	Django Web: Undo	Admin triggers 'Undo Last Action'	Stack pop restores previous record in MySQL	Database state rolled back	PASSED
TC-13	Security: Access	Student attempts admin undo action	Access Denied: Administrative permission required	Unauthorized request blocked	PASSED

9.2 User Interface Walkthrough & Live System Screenshots

The following sections demonstrate the live system interfaces captured during operational test execution across the responsive web platform and companion desktop demonstrator:

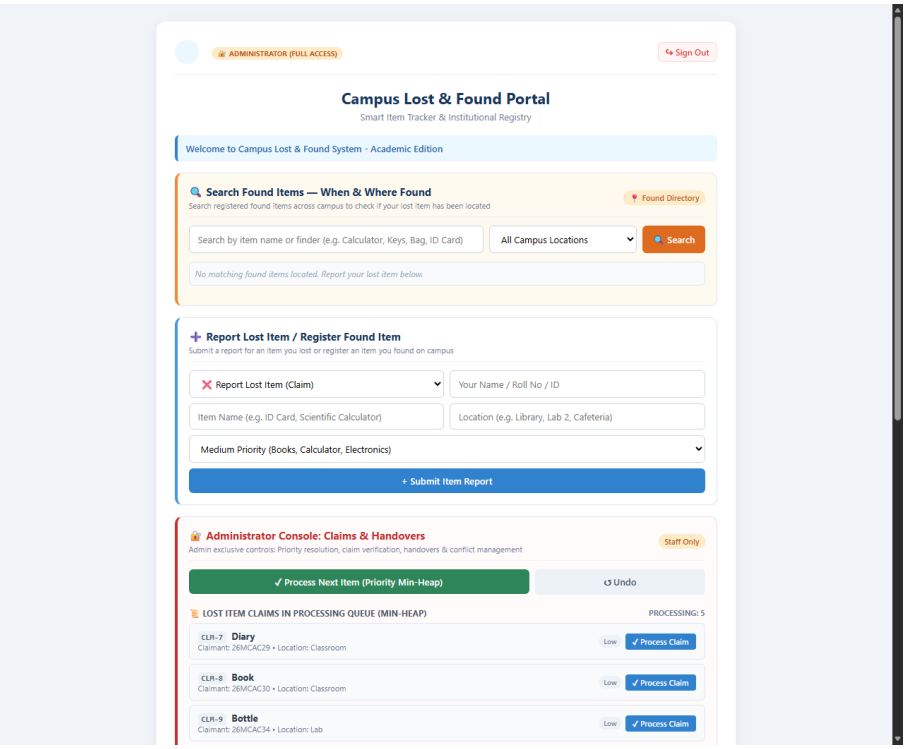


Figure 9.1: Web Application Dashboard: Public Item Catalog, Priority Queue & Activity Log

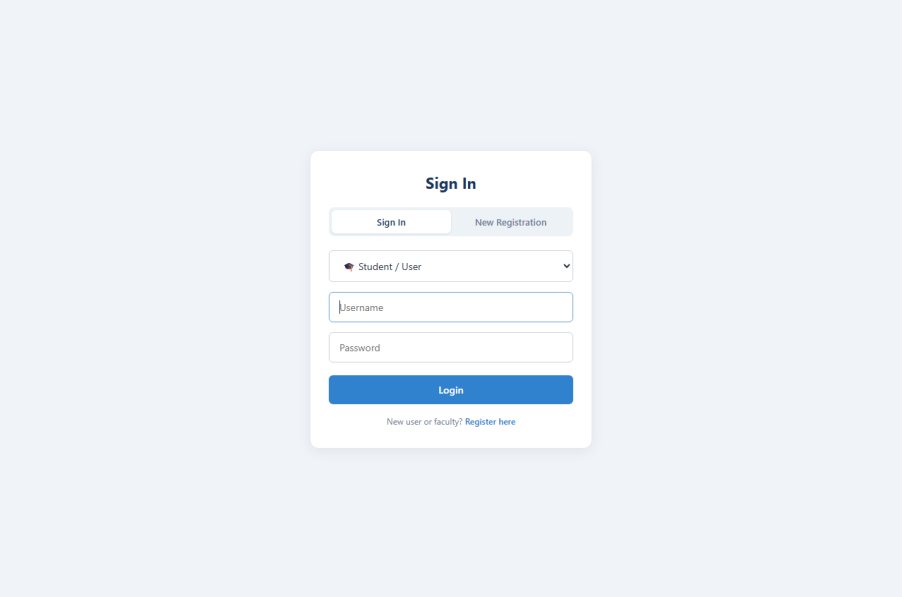


Figure 9.2: Web Application Multi-Role User Authentication Screen

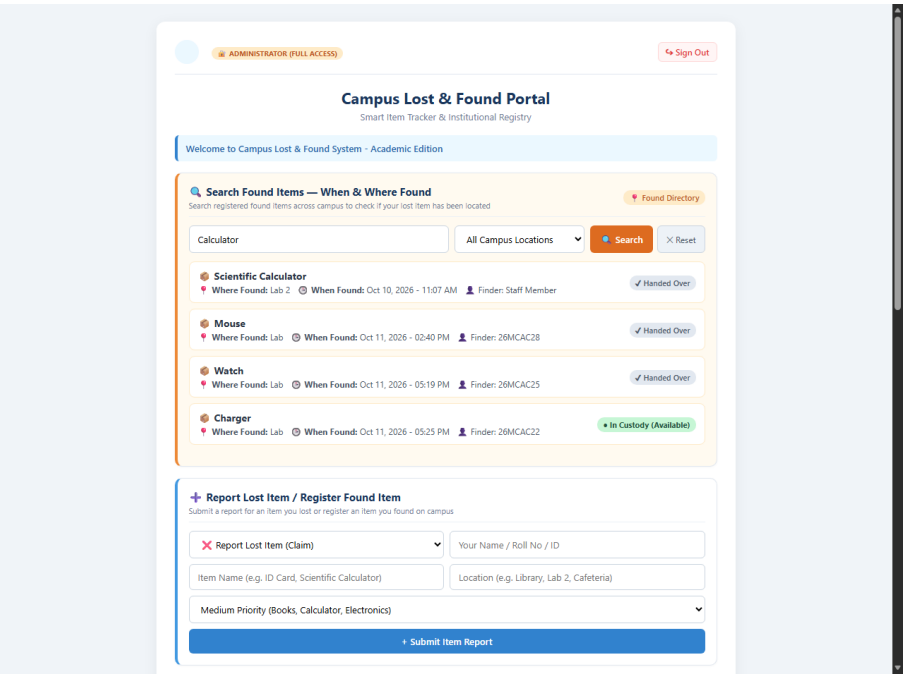


Figure 9.3: Web Application Location Proximity & Keyword Filtering Results

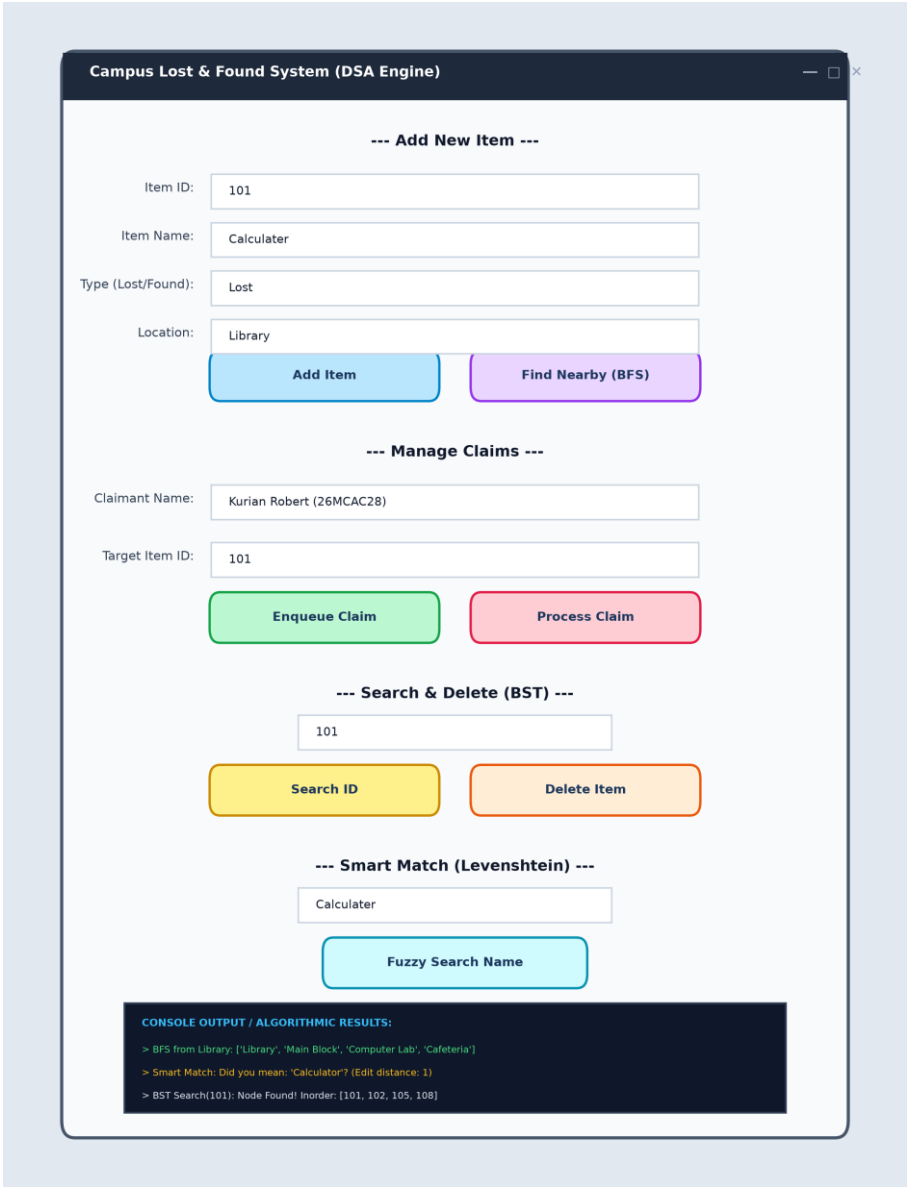


Figure 9.4: Desktop DSA Demonstrator: Tkinter Native Interface & Algorithmic Console

10. Future Enhancements

Although the current implementation satisfies all academic requirements for the 1st Semester MCA Data Structures and Algorithms curriculum, several practical extensions can be explored in future project phases:

- **1. Computer Vision Image Matching:** Integrating Convolutional Neural Networks (CNNs) to compare photographs of recovered items against lost item reports, calculating visual feature similarity embeddings.
- **2. Automated SMS & WhatsApp Alerts:** Connecting campus notification gateways to send automated text messages to students the moment an article matching their loss report is handed in.
- **3. Smart Locker RFID Integration:** Connecting the software to physical electronic lockers equipped with student RFID smart card scanners to enable secure 24/7 self-service pickup without requiring office staff.
- **4. Mobile Application with Geofencing:** Building a mobile application that uses campus Wi-Fi or GPS geofencing to notify students when they enter a building where their lost property is currently stored.

11. Conclusion & Technical Defense

11.1 Project Conclusion & Core Deliverables

The “Campus Lost and Found Management System with Priority Claim Scheduling” demonstrates how foundational Data Structures and Algorithms can be practically applied to solve a real logistics challenge at Kristu Jayanti University. By completing all four algorithmic phases—Linear Structures (Linked Lists, Stacks, Queues), Hierarchical and Graph Structures (Binary Search Trees and Campus BFS), Search and Sorting Optimizations (Linked List Merge Sort), and Dynamic Programming with Priority Queues (Levenshtein Distance and Min-Heap)—this project provided valuable hands-on experience in algorithm design and full-stack software development.

The final system successfully transitions campus property tracking from slow, handwriting-dependent paper registers into a searchable digital platform. It ensures fair and prioritized handling of urgent examination credentials, helps students recover items despite spelling mistakes, and maintains full administrative accountability through an audit log with undo capabilities.

11.2 Viva-Voce Architectural Defense Q&A

To support final examination defense, key architectural questions and model answers are summarized below:

- **Q1: Why is Merge Sort preferred over QuickSort for Singly Linked Lists?** Answer: QuickSort relies on O(1) random index access to choose pivots and swap elements, which requires costly O(n) pointer traversals on singly linked lists. Merge Sort divides linked lists using fast and slow pointer bisection and merges them in-place with O(1) auxiliary pointer memory, guaranteeing O(n log n) time in all cases.
- **Q2: How does the Dynamic Programming Levenshtein Distance matrix avoid redundant calculation?** Answer: It breaks down string comparison into overlapping subproblems. By recording intermediate transformation costs in a 2D matrix DP[i][j], optimal edit costs for sub-prefixes are reused in O(1) time rather than being recalculated exponentially in O(3^(m+n)) recursive calls.
- **Q3: Why use a Min-Heap instead of a sorted list for claim scheduling?** Answer: Inserting into a sorted list or array requires O(n) element shifts. A binary min-heap performs both insertions (`heappush`) and extractions (`heappop`) in logarithmic O(log n) time, ensuring constant-time access to the most urgent claim while scaling smoothly as campus records increase.

11.3 Academic References & Bibliography

The design, algorithmic formulation, and implementation of this dissertation drew upon established textbooks, peer-reviewed research papers, and technical documentation:

Ref ID	Citation / Source Details	Domain / Subject Relevance
[1]	Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to Algorithms (4th ed.). MIT Press.	Theoretical foundation for Binary Search Trees, Merge Sort, and Min-Heap Priority Queues.
[2]	Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady, 10(8), 707-710.	Original mathematical formulation of string edit distance dynamic programming recurrence.
[3]	Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2013). Data Structures and Algorithms in Python. John Wiley & Sons.	Implementation patterns for linked lists, stacks, queues, and graph traversals in Python.
[4]	Django Software Foundation. (2026). Django Documentation: Model-View-Template Architecture & ORM. https://docs.djangoproject.com/	Official architectural specifications for Django models, views, and authentication.
[5]	Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.	Relational database design, Third Normal Form (3NF) normalization, and ACID transaction rules.
[6]	Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.	Comprehensive reference for Python object model, memory management, and Tkinter GUI architecture.
[7]	Russell, S., & Norvig, P. (2020). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.	Heuristic search, priority queues, and graph traversal methodologies.

--- END OF PRACTICAL REPORT ---